Lecture 2 — Extending, constraining and recovering

Retour à « Lectures »

Lecture 2 — Extending, constraining, and recovering

4 September · 1h30 · full cohort · immediately followed by Lab 2

Serves: C2 (first half) and C3 (second half). **Why one lecture for two competencies:** there is no third lecture. C3's core is genuinely one idea, and it is better delivered sharply than padded.

This is the lecture, written down: one section per slide, the picture or table that was on screen, and what was said about it. Read it after the room if you were there, and instead of the room if you were not.

Part A — Extending and constraining

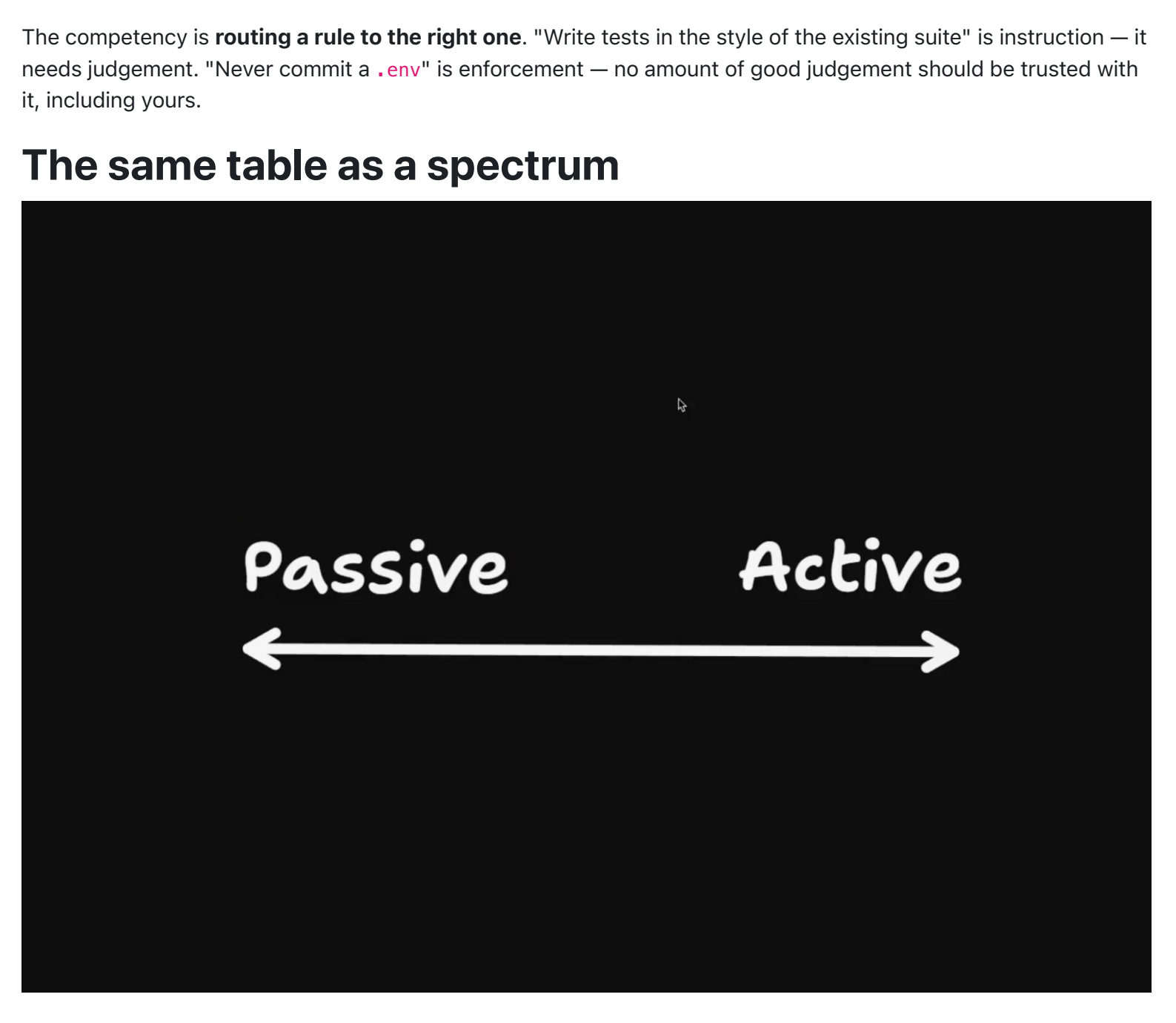
One idea to land: *instruction is persuasion; enforcement is physics. Know which one your rule needs.*

Two kinds of control

	Instruction	Enforcement
Example	`SKILL.md`, `AGENTS.md`, Cursor rule	hook, pre-commit, CI check
Fires because	the agent chose to reach it	it runs, regardless of the model
Failure mode	not reached, or reached and ignored	too rigid, blocks legitimate work
Good for	judgement, method, vocabulary	non-negotiables

The competency is **routing a rule to the right one**. "Write tests in the style of the existing suite" is instruction — it needs judgement. "Never commit a `.env`" is enforcement — no amount of good judgement should be trusted with it, including yours.

The same table as a spectrum



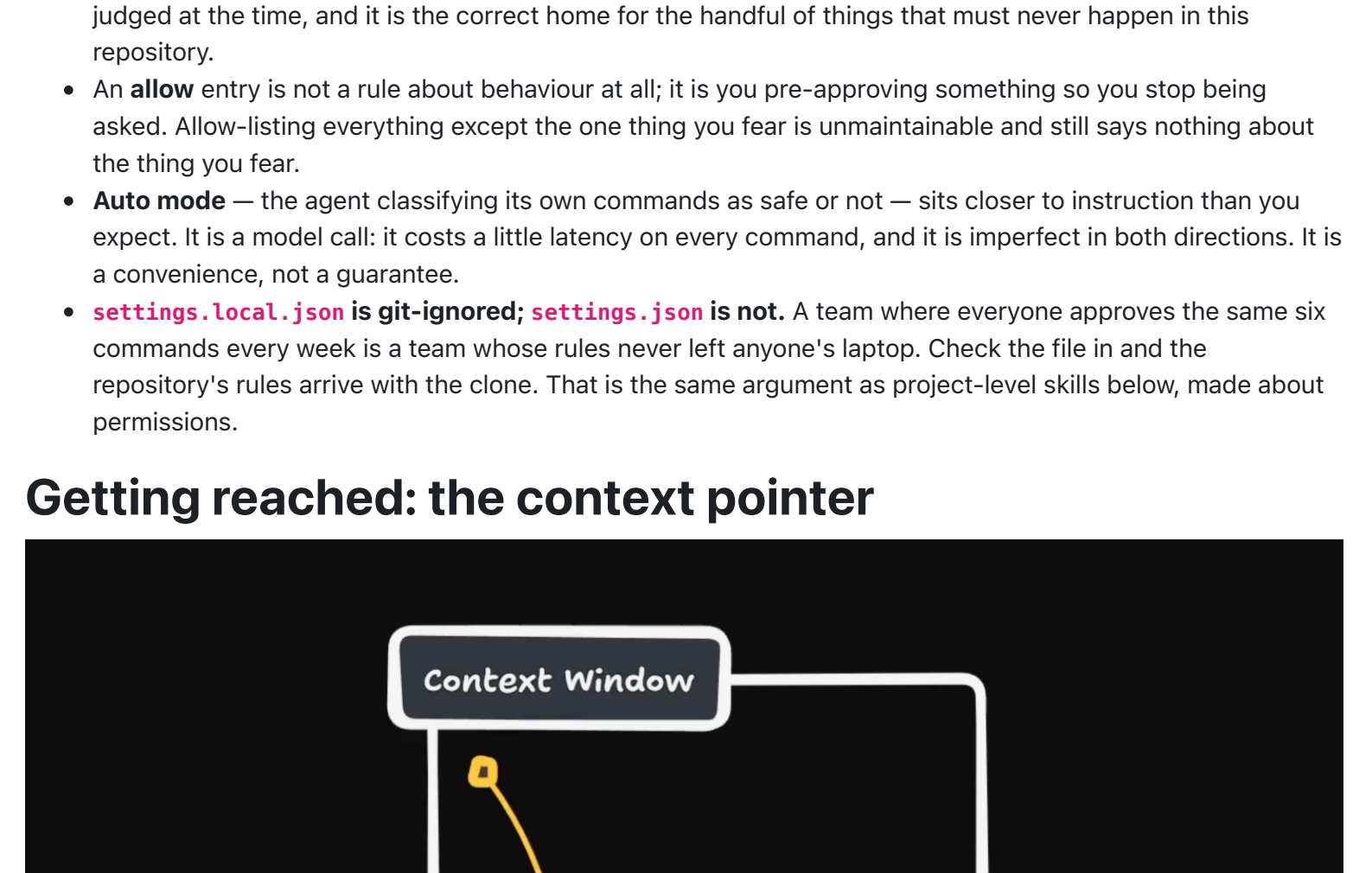
This is the more honest picture. At the passive end, something sits available and the agent decides whether to reach for it. At the active end, something runs whether or not the agent agrees. Most of what you write lands in the middle, and the middle is where rules quietly stop working — a document that is "usually" reached is passive, however emphatically it is worded.

Permissions: the surface in between

Every harness ships an allow/deny list — in Claude Code, `settings.json`. It is the one control you already have and never configure, and it teaches the spectrum better than the table does.

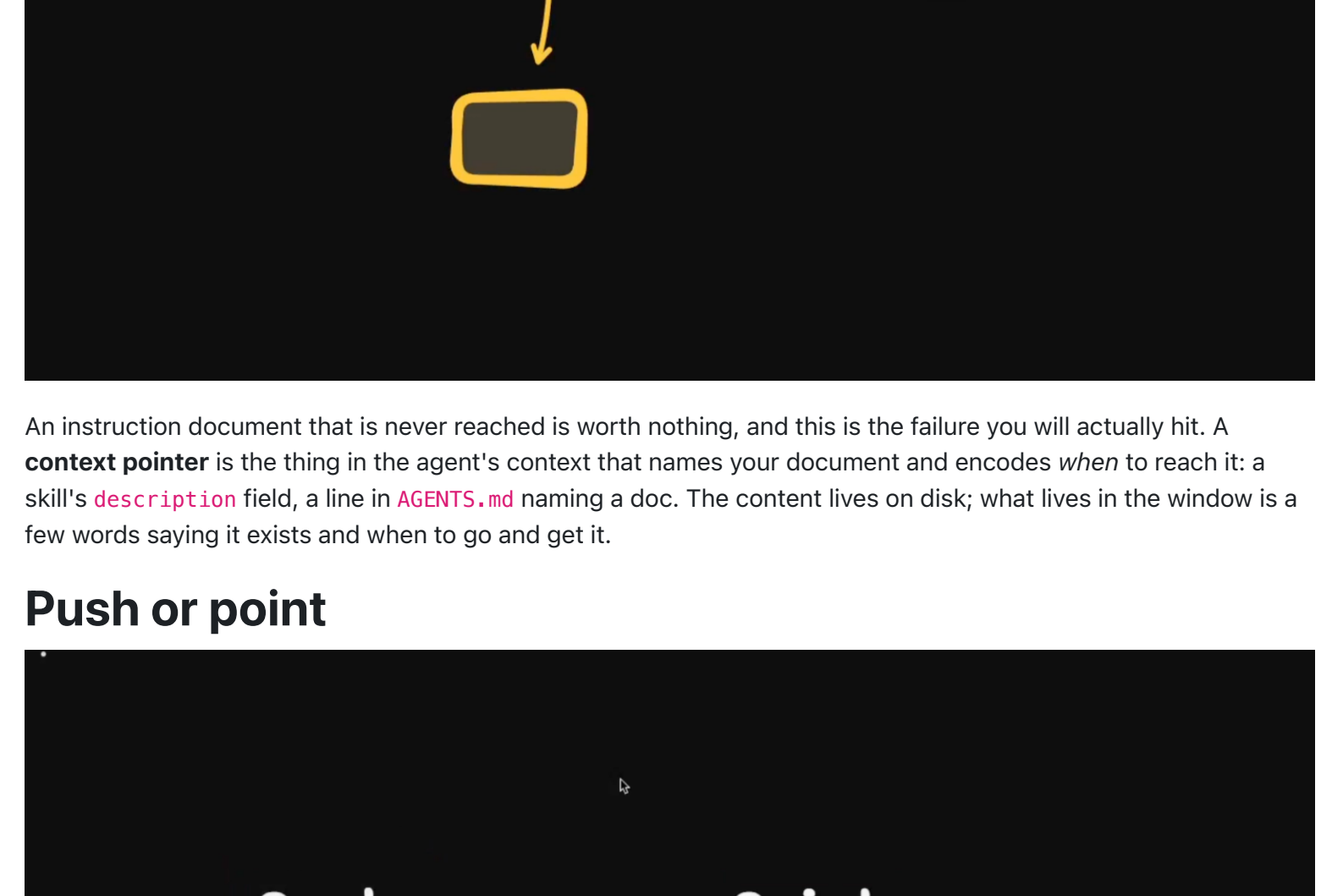
- A **deny** entry — `Bash(git push *)` — is enforcement. It stops the command outright, whatever the model judged at the time, and it is the correct home for the handful of things that must never happen in this repository.
- An **allow** entry is not a rule about behaviour at all; it is you pre-approving something so you stop being asked. Allow-listing everything except the one thing you fear is unmaintainable and still says nothing about the thing you fear.
- **Auto mode** — the agent classifying its own commands as safe or not — sits closer to instruction than you expect. It is a model call: it costs a little latency on every command, and it is imperfect in both directions. It is a convenience, not a guarantee.
- **`settings.local.json` is git-ignored; `settings.json` is not.** A team where everyone approves the same six commands every week is a team whose rules never left anyone's laptop. Check the file in and the repository's rules arrive with the clone. That is the same argument as project-level skills below, made about permissions.

Getting reached: the context pointer



An instruction document that is never reached is worth nothing, and this is the failure you will actually hit. A **context pointer** is the thing in the agent's context that names your document and encodes *when* to reach it: a skill's `description` field, a line in `AGENTS.md` naming a doc. The content lives on disk; what lives in the window is a few words saying it exists and when to go and get it.

Push or point



Push is always on — it is in every request, on every turn, whether the task needs it or not. **Point** is on demand. Both are legitimate: a five-line convention that applies to everything you do belongs pushed into `CLAUDE.md`; a 300-line runbook for one situation belongs pointed at. The failure mode in this room will be pushing — an `AGENTS.md` that has grown to cover everything and is therefore read carefully for nothing. Look at how big your own `CLAUDE.md` is, and let the number do the arguing.

The other kind of pointer: one that names code

A **navigation pointer** is one line in `AGENTS.md` naming a file the agent would not find by searching, and saying why it matters: *"`db/seed.py` — every schema change has to land here too; it is not referenced from the migrations."* The failure it fixes is specific: the agent greps, finds nothing, edits the four files it did find, and the fifth silently rots. Pasting the file's contents into `AGENTS.md` fixes it at the cost of shipping that file in every request forever; telling the agent to "search more carefully" fixes nothing, because searching is the thing that already failed. One bullet, naming the file and the situation, is the whole intervention.

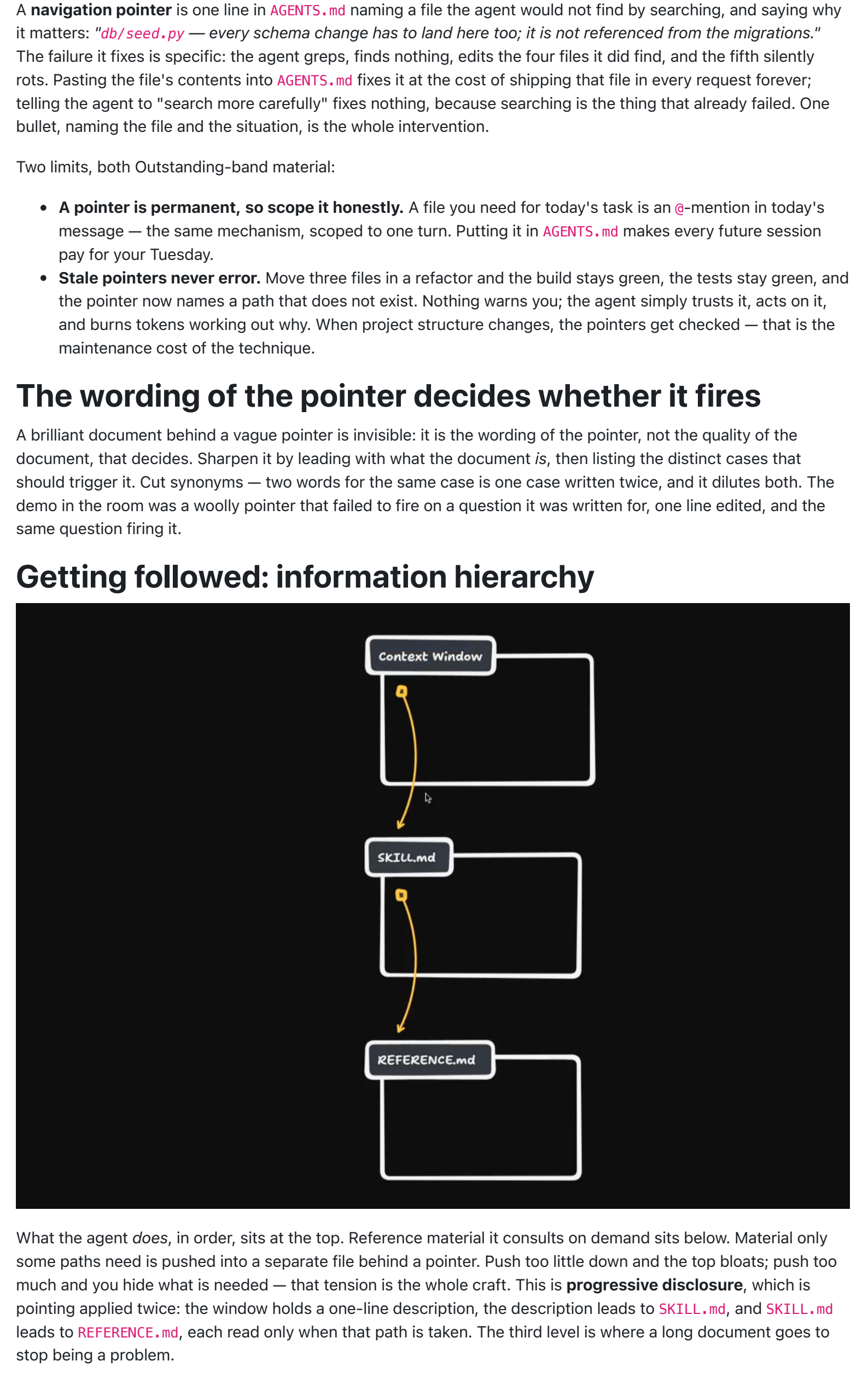
Two limits, both Outstanding-band material:

- **A pointer is permanent, so scope it honestly.** A file you need for today's task is an `@`-mention in today's message — the same mechanism, scoped to one turn. Putting it in `AGENTS.md` makes every future session pay for your Tuesday.
- **Stale pointers never error.** Move three files in a refactor and the build stays green, the tests stay green, and the pointer now names a path that does not exist. Nothing warns you; the agent simply trusts it, acts on it, and burns tokens working out why. When project structure changes, the pointers get checked — that is the maintenance cost of the technique.

The wording of the pointer decides whether it fires

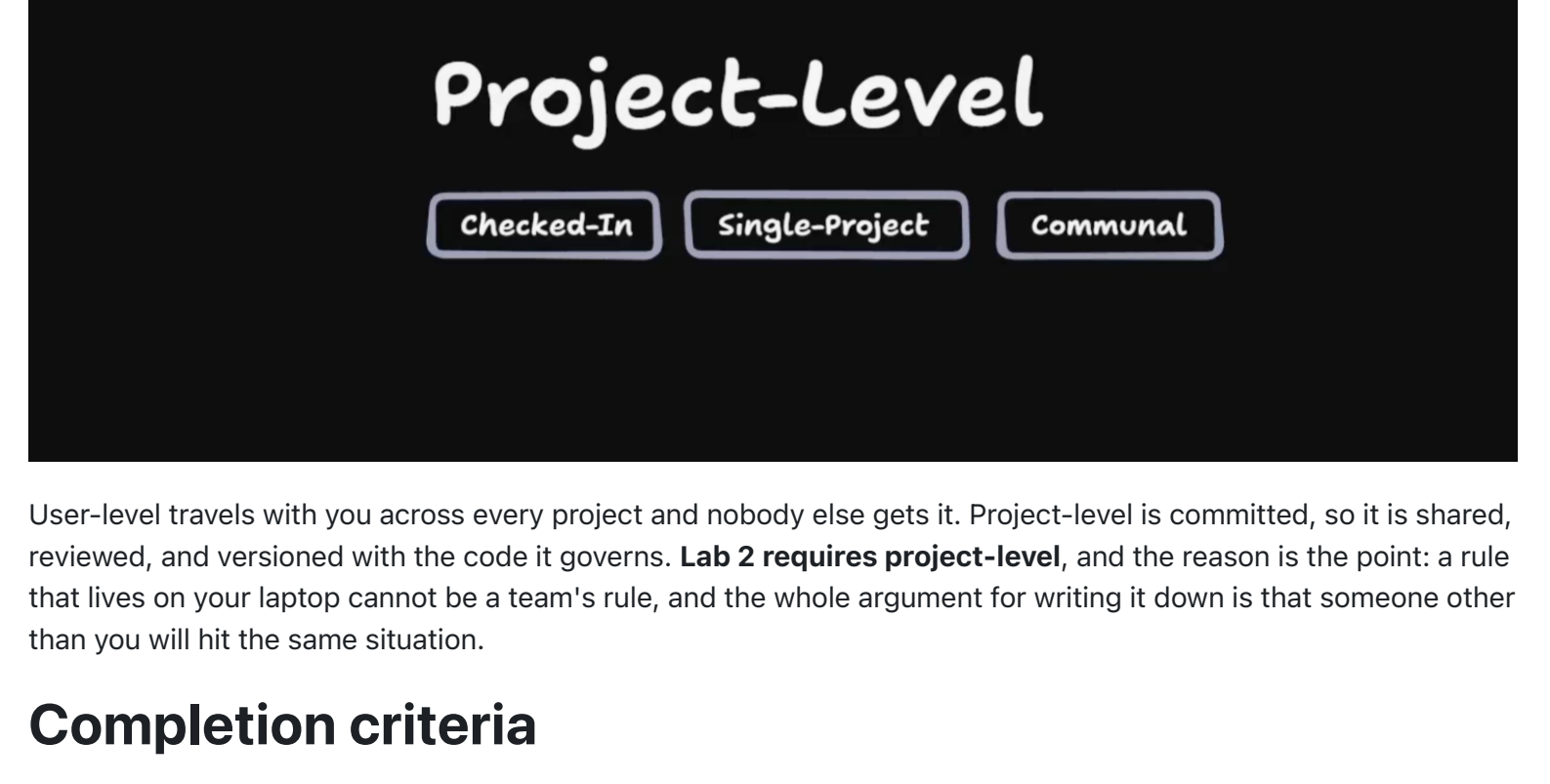
A brilliant document behind a vague pointer is invisible: it is the wording of the pointer, not the quality of the document, that decides. Sharpen it by leading with what the document *is*, then listing the distinct cases that should trigger it. Cut synonyms — two words for the same case is one case written twice, and it dilutes both. The demo in the room was a woolly pointer that failed to fire on a question it was written for, one line edited, and the same question firing it.

Getting followed: information hierarchy



What the agent *does*, in order, sits at the top. Reference material it consults on demand sits below. Material only some paths need is pushed into a separate file behind a pointer. Push too little down and the top bloats; push too much and you hide what is needed — that tension is the whole craft. This is **progressive disclosure**, which is pointing applied twice: the window holds a one-line description, the description leads to `SKILL.md`, and `SKILL.md` leads to `REFERENCE.md`, each read only when that path is taken. The third level is where a long document goes to stop being a problem.

Where the document lives is part of the design



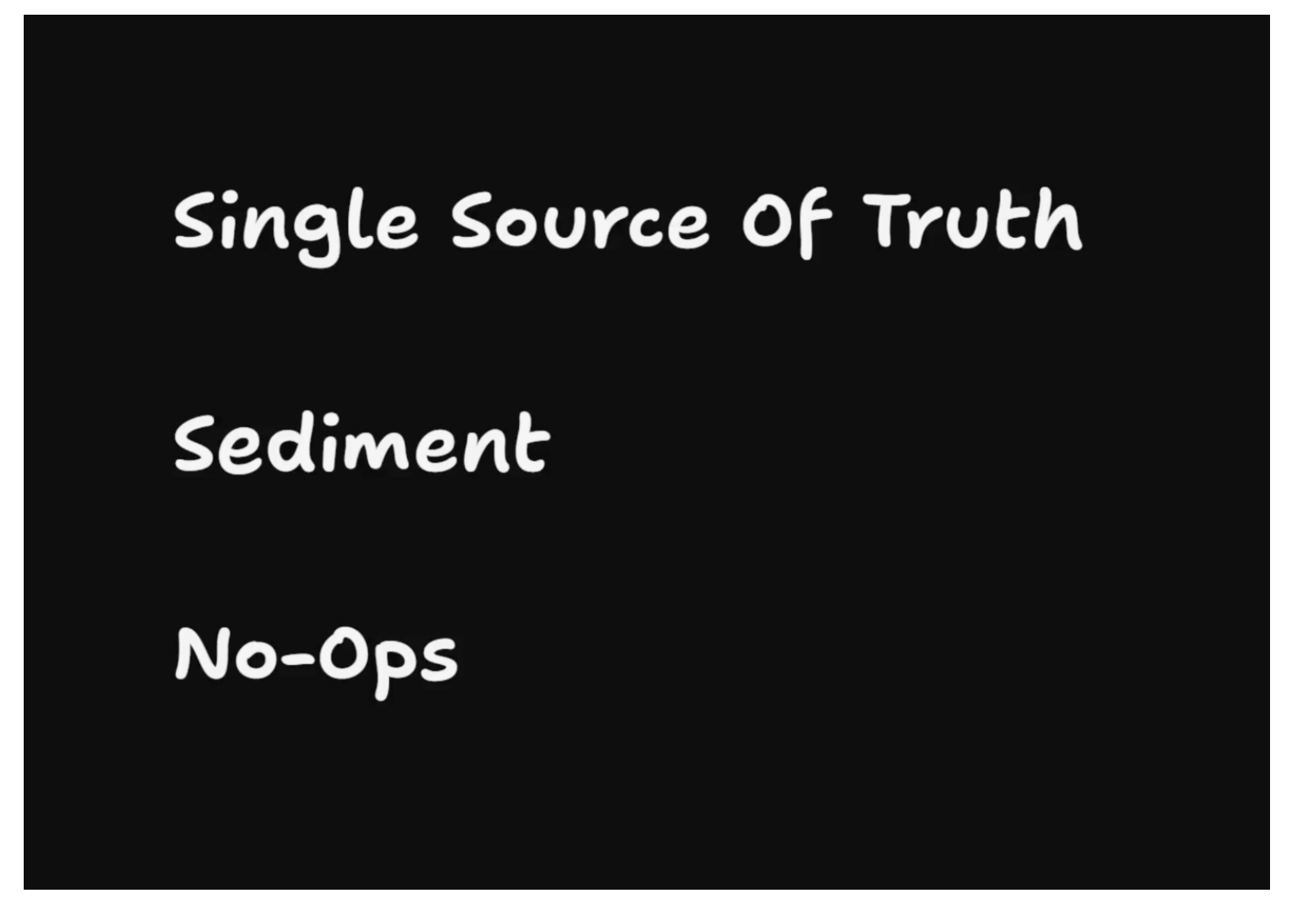
User-level travels with you across every project and nobody else gets it. Project-level is committed, so it is shared, reviewed, and versioned with the code it governs. **Lab 2 requires project-level**, and the reason is the point: a rule that lives on your laptop cannot be a team's rule, and the whole argument for writing it down is that someone other than you will hit the same situation.

Completion criteria

Every step ends on a condition telling the agent it is done, and two properties matter. **Checkable** — can the agent tell done from not-done? "Understanding reached" cannot be checked, so the agent declares victory early; "every modified table has a migration" can. **Demanding** — "every modified model accounted for" produces real work; "produce a change list" produces a list.

The grid asks for a checkable completion criterion, and for you to know when your pointer does not fire. (Grid: C2 Solid, Outstanding.)

The memory trap



Every harness now ships a button labelled *remember this*, and it is the thing you will reach for instead of writing a document. Automatic memory files fail in three ways, and they compound.

- **Single source of truth.** The rule now exists in two places — the memory file and the document, or the memory file and the code. Neither one is authoritative, and the agent will happily follow the stale one.
- **Sediment.** The file only ever grows. Nothing is reviewed, because nothing was written deliberately, so it silts up with lines nobody remembers agreeing to.
- **No-ops.** Most of what accumulates changes no behaviour at all — *"be careful with migrations"* — but is pushed into every single request and paid for on every turn.

> **Memory is the easiest thing to add and the hardest thing to remove.** A skill you wrote on purpose can be deleted on purpose. A memory file that grew cannot, because nobody can tell which lines are load-bearing.

This is not an argument against `CLAUDE.md`. It is an argument for writing it by hand, keeping it short, and pointing at everything else. Memory is push, and it is push that nobody chose.

Pruning: delete the line and see if the behaviour changes

The same three failure modes come for documents you wrote on purpose — a hand-written `CLAUDE.md` silts up too, just more slowly. So:

> **Delete the line and see if the behaviour changes.** If it does not, it was never doing anything, and it stays deleted.

An instruction can be true, on topic, well written, and still not worth a token — *"write a detailed commit message"* deleted from a skill, and the commit messages come out identical, because the model was already doing it. That is a **no-op**, and the test for one is deletion, not argument. Two related cuts:

- **A rule that lost its subject gets cut.** A line guarding against a quirk in a library you swapped out last quarter does not become harmless when it becomes obsolete — it actively drags the agent toward a world that no longer exists, and you pay context load on every request while it does. Generalising it into advice about libraries in general does not save it; that produces a line that is true of everything and changes nothing, which is a no-op wearing a disguise.
- **A rule written in two places has no authoritative home.** The same convention in `CLAUDE.md` and, in slightly different words, on a docs page, means two things to keep in sync and no way for the agent to tell which copy is live. Matching the wording exactly does not fix it. Keep one copy and point at it from the other place — and note that saying it twice is not free emphasis: it makes the rule weigh more than its real importance and quietly drowns out the lines said only once.

This is the work of C2 that has no artifact. Nobody is graded on what they deleted, but a student who can say *"I cut three lines from my skill and checked the behaviour was identical"* is showing exactly the judgement the Outstanding band is asking for. Lab 2 makes it a step.

Enforcement in practice

Claude Code hooks are shell commands fired on events, able to **block** a tool call before it runs — the demo was `git-guardrails-claude-code` refusing a `git push --force`. The harness-neutral equivalents are pre-commit hooks and CI checks: a pre-commit hook that rejects a secret is exactly the same idea and works in every harness in this room. The trap to name explicitly: **enforcement you can disable by asking nicely is not enforcement.** If the agent can edit the hook, the hook is instruction wearing a costume.

Why couldn't you just have asked?

Lab 2 grades this answer, so here it is properly. The honest answer is not that the agent is unreliable or unintelligent. It is **non-determinism**: the same prompt, the same context and the same model do not produce the same output twice, so there is always a non-zero chance the agent does something weird and bad — and that probability does not go to zero as the model gets smarter. A model that behaved impeccably all week is a sample, not a guarantee.

> **Instruction is a probability. Enforcement is a floor.**

For anything where the tenth outcome costs more than the nine good ones saved, you do not want a good probability. You want the floor.

The grid requires proof each artifact *fired*: a transcript excerpt, a terminal capture. Screenshot as you go — reproducing it on 10 September is miserable.

Part B — Recovering from failure

One idea to land: *build the loop before you build the theory.*

Why hypotheses lose

How do you currently debug with an agent? The honest answer is *describe the symptom, let it guess, try the guess, repeat*. That fails because the agent is fluent: wrong hypotheses arrive with the same confidence as right ones, and you have nothing to distinguish them.

The move is to refuse to hypothesise until you have **one command that goes red on this bug**. With it, everything else — bisection, instrumentation, elimination — is mechanical. Without it, no amount of reading code will save you.

> Build the right feedback loop and the bug is 90% fixed. Spend disproportionate effort here. Be aggressive, be creative, refuse to give up.

What a right loop looks like

An analyst says the weekly active-customers figure on the August dashboard is wrong: the week of 18 August shows **12**, and their own count off the same export is **14**.

Unaided, an agent produces three hypotheses in ten seconds, all fluent and all plausible — the week boundary is off by a day, two spellings of one customer id are being counted twice, the timezone conversion pushes late-Sunday transactions into the next week. You cannot tell which is right, and neither can it.

The loop is three lines, and you can write it before opening the code:

```
# tests/test_weekly_bug.py
def test_active_customers_week_of_18_august():
    rows = read_rows("data/exports/august.csv")
    assert count_active_customers(rows, week_start=date(2026, 8, 18)) == 14
```

`pytest tests/test_weekly_bug.py` takes a second, goes red, and prints `assert 12 == 14`. Notice that **nothing was hypothesised to write it**: it needed the symptom, the input and the expected number, and the analyst's message contained all three.

Then shrink it. The real export is thousands of rows; replace it with nine you write by hand — `txn_id,customer_id,timestamp,store_id,amount` — where every row is there for a reason:

- **five ordinary rows**, five different customers, mid-week and mid-afternoon: the baseline that should always be counted;
- **two rows for one customer on two different days**, because the function counts customers, not transactions;
- **one row at 23:50 on Sunday 24 August**, sitting on the boundary the timezone hypothesis is about;
- **one row whose customer id is `c-0042` where another row says `C-0042`**, which is the two-spellings hypothesis on purpose.

Seven distinct customers, so the assertion becomes `== 7`, and it is red too. Nine is not an arbitrary number: it is four hypotheses plus a baseline.

Four properties make it the right loop, and they are what you are graded on:

- **It goes red on this bug.** The suite is green and stays green; this command is the only thing in the repository that knows the bug exists.
- **It asserts the symptom** — `== 7`, not "returns a number" and not `!= 5`. A loop that only demands the number *change* goes green on 6, and 6 is still wrong.
- **It is deterministic and cheap.** Nine rows on disk, a fixed `week_start`, no warehouse, no clock, no network.
- **It makes the hypotheses separable.** Push the Sunday row to 22:00 and rerun: if the number moves it is the timezone, and if it does not, that hypothesis is dead. Fix the casing and rerun. Each hypothesis is one edited row and a one-second rerun — that is what "mechanical" means.

Compare the loop most people already have: `python -m dashboard rebuild && python -m dashboard weekly` over the real export. Forty seconds, rebuilds the warehouse, prints a table you read with your eyes, and it never fails — nothing errored, the number is just wrong on screen and it is on you to spot it.

The nine rows were the expensive part, not the assertion. Deciding what each row is for is the effort the quote above is asking for. It feels like work before the real work. It is the real work.

Building a loop

Roughly in order of preference — and it helps to read the list as one bug, not seven techniques: these are **seven ways to get a red on the same weekly-customers bug**. Go down the list until one is cheap in the system you actually have.

1. **A failing test** at whatever seam reaches the bug — the one above:

   ```
   pytest tests/test_weekly_bug.py   # assert 12 == 14
   ```

2. **A curl or HTTP script** against a running dev server, when the bug lives behind the API and finding the seam is more work than hitting the endpoint:

   ```
   curl -s 'localhost:8000/api/weekly?week=2026-08-18' | jq -e '.active_customers == 14'
   ```

   `jq -e` is the trick: it exits non-zero when the check fails, so you get red or green instead of something to read.

3. **A CLI invocation** on a fixture, diffed against known-good output — best when the answer is a table rather than a number:

   ```
   python -m dashboard weekly --input tests/fixtures/week_18_aug.csv > /tmp/got.csv
   diff -u tests/fixtures/week_18_aug.expected.csv /tmp/got.csv
   ```

   The diff is the assertion, and it tells you which rows moved.

4. **A replayed trace** — save one real payload to disk and push it through the code path in isolation:

   ```
   # scratch/replay.py
   batch = json.load(open("scratch/2026-08-24-batch.json"))   # captured from the real run
   print(count_active_customers(ingest_batch(batch), week_start=date(2026, 8, 18)))
   ```

   This is the move when the bug only happens on production data: you stop needing production to run.

5. **A throwaway harness** — the minimum subset of the system that reaches the bug in one function call, with no app, no config and no database:

   ```
   # scratch/repro.py — 10 lines, deleted on Friday
   from dashboard.metrics import count_active_customers
   rows = [{"customer_id": "C-0042", "timestamp": "2026-08-24T23:50:00"}, ...]
   print(count_active_customers(rows, week_start=date(2026, 8, 18)))
   ```

   This file is allowed to be ugly and is not going in the pull request.

6. **A property or fuzz loop** — for *sometimes wrong*, run a thousand inputs and look for the shape of the failures:

   ```
   for _ in range(1000):
       rows = random_rows()                          # random ids, timestamps across the week
       expected = len({r["customer_id"].upper() for r in rows if in_week(r)})
       assert count_active_customers(rows, week_start=WEEK) == expected, rows
   ```

   What you want is not the first failure but the **pattern** across failures: if every counter-example contains a Sunday-evening timestamp, you have the cause without reading the implementation.

7. **A bisection harness** — if it appeared between two known-good states, automate "boot at X, check, repeat". Once you have number 1, it is one command:

   ```
   git bisect start HEAD v2.3.0
   git bisect run pytest tests/test_weekly_bug.py
   ```

   That is the payoff for the whole exercise: the loop is what makes bisection free.

For **data pipelines specifically** — which is what you will be working in — the highest-value loops are numbers 1 and 3, in two shapes. A fixture file plus an assertion on the output rows, which you have just seen. And a **differential run**: the same input through two configurations, diffed against each other rather than against a known-good answer.

```
diff <(python -m dashboard weekly --input fixture.csv --tz=UTC) \
     <(python -m dashboard weekly --input fixture.csv --tz=Europe/Paris)
```

That last one is the loop nobody invents on their own, and it is worth the space: **you do not need to know the right answer to get a red.** You need two runs that ought to agree — two timezones, two config flags, the old code and the new — and any difference between them is a finding. That is the version you will use in industry.

Tightening the loop

Treat the loop as a product. Once you have one, make it **faster** (cache setup, skip unrelated initialisation, narrow the scope), **sharper** (assert on the specific symptom — the number you were given, not a weaker relative of it: *"it doesn't crash"* is not an assertion, `!= 5` goes green on 6 and 6 is still wrong, and only `== 7` stays red until the bug is gone; this is the one that separates Basic from Solid, and a loop that is merely dissatisfied with the current answer will accept the next wrong one), and **more deterministic**.

Determinism has three sources, and all three show up in data work. **Pin the clock** — freeze `now()` to a fixed instant, or a loop whose answer depends on the day you run it goes green on its own next Monday. **Seed the random number generator** (`random.seed(0)`) so that anything sampled, shuffled or generated is identical on every run: a fixture built with `random.sample`, a fuzz loop like number 6 above, a pipeline that samples rows for a spot check. A counter-example you cannot regenerate is not evidence — with a seed, you can hand someone the number and they get your failure. **Isolate the filesystem** — a temp directory per run, built from the fixture, never the warehouse sitting in your working copy carrying yesterday's state.

A 30-second flaky loop is barely better than nothing. A 2-second deterministic one is a superpower. For intermittent bugs the goal is not a clean reproduction but a **higher reproduction rate**: loop it 100 times, parallelise, inject sleeps. 50% is debuggable; 1% is not.

Elimination, not inspiration

Each hypothesis becomes a change to the loop that would distinguish it. Ruling things out *is* the progress. **The grid asks you to name one cause you eliminated** — so keep track as you go.

The fix is not the end either. The regression test must fail on the original code: run it against the old code and watch it go red, or you do not know it tests anything. And test at a seam — a regression test coupled to internals dies at the next refactor and stops protecting you.

Route the finding

A bug is evidence about the system, not just about one line:

- the specification was ambiguous → sharpen the spec,
- the agent keeps making this mistake → an instruction document (**C2**),
- it cannot be locked down because there is nowhere to test → the structure is wrong,
- it should never have been possible → enforcement in CI (**C2** again),
- it is a matter of judgement that no check can express → **the review**.

The destination nobody thinks of: the review

An automated check is deterministic and effectively free. A linter, a type check, a test suite gives the same verdict every run and costs nothing to run again. That is precisely why it cannot deliver judgement: a rule a linter can enforce is a rule that never needed writing down in prose.

An automated review is qualitative and costs tokens. It is an agent reading a diff against your standards and forming an opinion. It catches the class of thing no check can express — *this belongs in the other module*, *this duplicates the thing next to it*.

And the placement argument, which is the useful part: **standards belong where the review reads them, not pushed into `CLAUDE.md`.** An implementation session is already carrying three jobs in one window — explore, write, debug — and a long conventions document competes with all three for attention. A review session carries none of them and has room to spare. A `CODING_STANDARDS.md` that the review skill reads costs nothing on the turns where nobody is reviewing. Same rule, same words, radically different price.

Close the loop back to Part A

The best answers in the oral connect C3 to C2: *"this class of bug should have been blocked, and here is the check that would have done it."* That is the Outstanding band.

The bug you have not met yet

On 8 September you get a repository everyone else also gets, with **one bug**, and a **90-minute slot** to do it in. It is frozen with everything else at **20:00 on 10 September**, and you present on the 11th. The brief is Autonomy 2 — C3 Exercise.

One warning: **the test suite passes. It will keep passing.**

The pictures in this lecture are from the AI Hero crash course.

Modifié le: jeudi 3 septembre 2026, 18:25